Ricardo Marcelo Suman Júnior

Balanceamento de carga em jogos online baseado na Área de Interesse

São Paulo – Brasil

2019

Ricardo Marcelo Suman Júnior

Balanceamento de carga em jogos online baseado na Área de Interesse

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Fábio de Toledo Pereira

São Paulo – Brasil

2019

Resumo

Jogos multijogador e massivos permitem a interação entre milhares de jogadores concorrentes online. Devido a necessidade de prover uma melhor experiência e interação entre os jogadores, o balanceamento de carga é necessário nos servidores que abrigam tais jogos. Uma técnica bastante utilizada é de particionamento do mundo virtual baseado em área de interesse, que consiste na divisão do mundo em áreas e cada área é administrada por um servidor, partindo do pressuposto de que o jogador está interessado apenas nos eventos ocorrendo próximos a ele.
Neste presente trabalho, é proposta uma arquitetura baseada em conceitos de sistemas distribuídos, com o objetivo de objetivo de obter um servidor escalável e com uma latência reduzida. Para isso, foi desenvolvido um cliente de exemplo com mecânicas básicas de jogos massivos online e dois servidores. Um servidor não distribuído e outro seguindo a arquitetura proposta para realização de uma análise comparativa entre ambas soluções. A análise comparativa foi feita através da medição de latência e taxa de transferência nas duas abordagens.

Palavras-chaves: jogos online, mmo, sistemas distribuídos, aoi, área de interesse, latência, taxa de transferência

Abstract

Multiplayer and massive games allow interaction between thousands of concurrent players online. Due to the need to provide a better experience and interaction between players, load balancing is required on servers hosting such games. A widely used technique is partitioning the virtual world based on area of interest, which consists of dividing the world into areas and each area is managed by a server, assuming that the player is only interested in the events occurring next to him.
In this work, an architecture based on distributed systems concepts is proposed, with the objective of obtaining a scalable server and with a reduced latency. For this, an example client was developed with basic mechanics of massive online games and two servers. One non-distributed server and another following the proposed architecture to carry out a comparative analysis between both solutions. The comparative analysis was done through the measurement of latency and throughput in the two approaches.

Keywords: online games, mmo, distributed systems, aoi, area of interest, latency, throughput

Sumário

Lista de ilustrações **6**

1 INTRODUÇÃO **8**
1.1 Contextualização **8**
1.2 Objetivo **9**
1.3 Objetivos específicos **9**

2 REVISÃO BIBLIOGRÁFICA **11**
2.1 Sistemas Distribuídos **11**
2.1.1 Arquiteturas 12
2.1.2 Escalabilidade 12
2.1.3 Balanceamento de carga 14
2.1.4 Comunicação 15
2.1.5 Consistência e Replicação 17
2.2 Arquitetura de jogos online **18**
2.2.1 Game Loop 18
2.2.2 Mundo Persistente 18
2.2.3 Servidor Autoritativo 18
2.2.4 Gerenciamento de área de interesse 19
2.2.5 Subdivisão de servidores 20
2.2.6 Algoritmos de particionamento 21
2.3 Medição de performance de servidores **22**
2.4 Trabalhos relacionados **24**
2.4.1 A Distributed Architecture for Massive Multiplayer Online Role-Playing Games 24
2.4.2 Distributed Architectures for Massively-Multiplayer Online Games 24

3 DESENVOLVIMENTO **26**
3.1 Primeira fase - Cliente **26**
3.2 Segunda fase - Servidor não-distribuído **27**
3.3 Terceira fase - Servidor distribuído **27**
3.4 Arquitetura proposta **29**

4 RESULTADOS **33**
4.1 Primeira parte **33**
4.2 Segunda parte **35**
4.3 Análise **36**

5 CONCLUSÃO 38

REFERÊNCIAS 39

Lista de ilustrações

Figura 1 – Arquitetura cliente-servidor 13
Figura 2 – Área de interesse 20
Figura 3 – Geração do quadtree a partir de uma grade 2D 21
Figura 4 – Geração do kD-Tree 22
Figura 5 – Characteristic Performance Graph's of a Server 23
Figura 6 – Solução não-distribuída 25
Figura 7 – Solução distribuída 25
Figura 8 – Cliente 27
Figura 9 – Zona 1 29
Figura 10 – Zona 2 29
Figura 11 – Arquitetura 30
Figura 12 – Filas (Queue) - Python 31
Figura 13 – Estrutura do objeto Player - JSON 31

Lista de abreviaturas e siglas

MMOG	Massively Multiplayer Online Game
MMORPG	Massively Multiplayer Online Role-Playing Game
RPC	Remote Procedure Call
MOM	Message-Oriented Middleware

1 Introdução

1.1 Contextualização

Jogos online são jogos que podem ser jogador por meio de dispositivos eletrônicos conectados na internet. Dentre os gêneros de jogos online, existe o gênero MMORPG (Massively Multiplayer Online Role-Playing Game), onde participam uma grande quantidade de jogadores que se conectam em um mundo virtual. O mundo virtual é dito persistente, pois mesmo que os jogadores se desconectem, o jogo continua rodando.

De acordo com (ACHTERBOSCH; PIERCE; SIMMONS, 2008), este gênero representa um setor de jogos de computador que cresceu muito ao longo dos anos, atingindo grandes quantidades de jogadores ao redor do mundo, desde a criação do primeiro MMO em 1996, Meridian 59 (VAULT, 2012). Um exemplo de MMORPG popular é Ragnarök Online, desenvolvido pela empresa sul-coreana Gravity Corp. Foi lançado em 31 de agosto de 2002 na Coreia do Sul e em setembro de 2004 no Brasil. A arte, cenários, mitos e lendas do jogo foram baseados no manhwa[0] Ragnarök de Lee Myung-jin. Na Coreia do Sul, Ragnarök Online foi o primeiro jogo online a ser exportado com sucesso para fora do país. E no Brasil, foi o primeiro jogo online a ter sido traduzido em português, rodando em servidores nacionais e contando até com um sistema de transações em Reais (R$) (JOGOS, 2011). Segundo (GAMESPOT, 2007), em 2007 foi estimado que o jogo possuía mais de 25 milhões de assinantes no mundo todo. Outro grande exemplo é World of Warcraft, também conhecido como WoW. É um MMORPG da produtora Blizzard que foi lançado em meados de 2004. É considerado o mais popular dos jogos do gênero, chegando a atingir a marca de 11,5 milhões de usuários ativos em 2008 com o lançamento de sua segunda expansão Wrath of the Lich King (LIKARISH et al., 2011) (COMTEX, 2008).

Tradicionalmente os jogos online do gênero MMORPG utilizavam a arquitetura cliente/servidor. No entanto, com o aumento da popularidade desses jogos, a quantidade de jogadores começou a aumentar cada vez mais, fazendo com que os desenvolvedores buscassem aprimoramentos com o objetivo de abrigar mais jogadores concorrentes mantendo a qualidade da experiência do jogador. O uso da arquitetura cliente/servidor traz diversas vantagens. Segundo (BARRON, 2001), a principal vantagem oferecida por esta arquitetura é a segurança, pois toda a lógica do jogo fica centralizada no servidor e não nos clientes, afinal estas informações podem ser interceptadas e modificadas por usuários com más intenções. Uma outra importante vantagem citada por (GAUTHIERDICKEY, 2014) é que

[0] manhwa: (em coreano: 만화, transl. man-hwa, literalmente "história(s) em quadrinhos") é um termo geral coreano para designar histórias em quadrinhos. Fora da Coreia do Sul, o termo se refere especificamente a histórias em quadrinhos da Coreia do Sul.

como as informações estão centralizadas no servidor, o conflito de informações é menor. Contudo, caso essa abordagem seja utilizada com um único servidor, observa-se algumas desvantagens. Dentre as desvantagens, também citadas por (GAUTHIERDICKEY, 2014) estão o delay, que representa a diferença de tempo entre o envio e recebimento de uma informação. Uma vez que todas mensagens enviadas pelos jogadores são encaminhadas para um único servidor centralizado e também o tráfego de dados que aumenta conforme o número de jogadores cresce, causando congestionamento no servidor.

Diversas soluções foram propostas utilizando conceitos de sistemas distribuídos de modo a manter a segurança oferecida pela arquitetura cliente/servidor e melhorar sua performance. Como demonstrado por (ASSIOTIS; TZANOV, 2006), desenvolver um servidor de MMO utilizando a arquitetura cliente/servidor e seguindo a abordagem distribuída traz benefícios de performance e suporte à grandes quantidades de usuários concorrentes, sem sacrificar a eficiência e a segurança do servidor. Uma possível solução para reduzir o tráfego parte do pressuposto de que o jogador está interessado em enxergar apenas aquilo que está em seu raio de visão, ou seja, eventos acontecendo próximo a ele. Esse raio de visão também é chamado de área de interesse. Partindo deste princípio, é possível particionar o mundo virtual em pequenas áreas, e com a abordagem distribuída, atribuir um servidor gerenciando cada área.

Essa será a solução a ser estudada neste trabalho, onde é proposta a aplicação da técnica de particionamento do mundo virtual baseada na área de interesse junto com a abordagem distribuída, com o objetivo de obter um servidor escalável.

1.2 Objetivo

O objetivo deste trabalho é aplicar a técnica de balanceamento de carga baseada em área de interesse, a fim de obter um servidor distribuído e escalável.

1.3 Objetivos específicos

Para se chegar no objetivo geral, será necessário atingir os seguintes objetivos específicos:

- Criar uma mecânica de jogo simples que englobe os principais aspectos de um jogo online e massivo para que possa ser utilizado como cliente de exemplo;
- Implementar o servidor do jogo seguindo a abordagem não distribuída;
- Implementar o servidor do jogo seguindo a abordagem distribuída, dividindo o mundo virtual em áreas com servidores responsáveis por cada uma;

- Coletar os resultados e comparar as duas abordagens, por meio da medição da latência e da taxa de transferência dos servidores.

Neste presente trabalho será utilizado a linguagem de programação Python 3.6 na implementação do cliente e do servidor. No cliente será utilizado biblioteca gráfica para jogos PyGame, na versão 1.9.6.

2 Revisão Bibliográfica

Neste capítulo, serão apresentados os fundamentos teóricos necessários para o entendimento deste trabalho.

Esta seção está organizada em três partes. A primeira subseção ***2.1*** trata de fundamentos de ***Sistemas Distribuídos***, a segunda subseção ***2.2*** sobre ***Arquiteturas de jogos online*** e a última ***2.3*** sobre ***Trabalhos relacionados***.

Na subseção ***2.1*** são abordados conceitos como ***Arquitetura***, ***Escalabilidade***, ***Balanceamento de Carga***, ***Comunicação*** e ***Consistência e Replicação de Dados*** em Sistemas Distribuídos. Tais conceitos são necessários para o compreendimento de como estas soluções funcionam e podem ser aplicadas em servidores de jogos online.

Na subseção ***2.2*** são abordados conceitos e requisitos referentes à desenvolvimento de jogos online. Dentre estes conceitos, estão ***Game Loop***, ***Servidor Autoritativo***, ***Gerenciamento de área de interesse***, ***Subdivisão de servidores*** e ***Técnicas de particionamento de mundos***. Estes conceitos são necessários para o entendimento do funcionamento e dos problemas enfrentados no desenvolvimento de jogos online.

2.1 Sistemas Distribuídos

De acordo com (TANENBAUM; STEEN, 2017), um sistema distribuído pode ser definido como "um conjunto de computadores independentes que se apresenta a seus usuários como um sistema único e coerente".

Diferentemente de sistemas centralizados onde toda uma aplicação ou serviços são executados em uma única máquina física, os sistemas distribuídos procuram dividir as cargas de trabalhos entre as diversas máquinas que os compõem. Sendo esta composição de máquinas chamada de cluster.

Um sistema distribuído não é constituído apenas de máquinas, é preciso que a aplicação a ser distribuída seja arquitedada e desenvolvida para funcionar sob um conjunto de máquinas. Tendo isto em vista, as arquiteturas ajudam a definir o funcionamento e a comunicação entre os componentes em sistemas distribuídos.

2.1.1 Arquiteturas

Segundo (TANENBAUM; STEEN, 2017), a complexidade presente em sistemas distribuídos se faz presente pelo fato de que nesse contexto, os componentes de um software estão dispersos em várias máquinas. Tal complexidade exige a definição de uma arquitetura para ditar o comportamento dos componentes que constituem o sistema distribuído.

De acordo com (COULOURIS et al., 2013), uma arquitetura de sistemas distribuídos define a relação entre os componentes e as responsabilidades de cada um. Com a arquitetura definida, é possível entender como os componentes estão organizados e como eles devem comunicar-se entre si.

Existem alguns tipos de arquiteturas, sendo a mais tradicional, chamada de ***arquitetura centralizada***. Essa arquitetura consiste em ter uma máquina, chamada de servidor, que provê informação para outras máquinas chamadas de clientes. Já a ***arquitetura descentralizada*** é a abordagem onde as máquinas envolvidas têm igual responsabilidade, todas servem e requisitam informações.

Arquitetura centralizada

Segundo (WETHERALL; TANEMBAUM, 2011), na arquitetura centralizada, também chamada de cliente-servidor, estão envolvidos dois processos. Um processo na máquina cliente e outro na máquina servidora. A comunicação entre os dois processos se dá início com o processo cliente mandando uma mensagem pela rede ao processo servidor, requisitando alguma informação. Então, o processo cliente aguarda por uma mensagem de resposta. Ao receber alguma mensagem, o processo servidor realiza algum tipo de trabalho solicitado ou busca pela informação requisitada e envia uma mensagem de resposta ao cliente. O funcionamento pode ser visualizado na Figura 1.

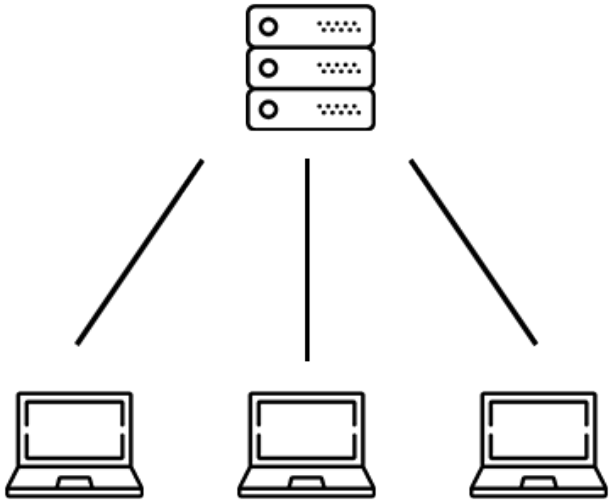
Arquitetura descentralizada

Segundo (COULOURIS et al., 2013), a arquitetura descentralizada funciona como uma espécie de rede colaborativa, onde os nós colaboram entre si de maneira uniforme. Todos os nós têm a mesma responsabilidade, todos atuam como consumidores e provedores, ou seja, clientes e servidores.

2.1.2 Escalabilidade

De acordo com (RAJAPAKSE, 2012), o termo ***escabalidade*** pode ser definido como "a facilidade em que o sistema ou componente pode ser modificado para se ajustar a um determinado problema". Um sistema que é escalável possui estas três simples

Figura 1 – Arquitetura cliente-servidor



Fonte: Autor

características:

- O sistema acomoda o aumento do uso de processamento;
- O sistema acomoda o aumento do fluxo de dados;
- O sistema é sustentável e funciona com desempenho razoável.

A escalabilidade de um sistema não envolve apenas a velocidade do mesmo, mas sim no preparo de crescimento do sistema conforme o uso, mantendo uma performance razoável. No geral, a escalabilidade de um sistema depende da natureza do mesmo. Em servidores web, por exemplo, a escalabilidade está em preparar o servidor para aguentar desde pequenas até grandes quantidades de conexões. De acordo com (RAJAPAKSE, 2012), a performance do sistema é afetada a ponto de ser perceptível ao usuário, pode ser um sinal de que o sistema não é escalável e consequentemente enfrentará grandes problemas conforme a quantidade de usuários aumenta.

Há dois tipos de escalabilidade existentes, que serão abordadas no próximo tópico, sendo elas: ***escalabilidade horizontal*** e ***escalabilidade vertical***.

Escalabilidade vertical

Podemos ver a escalabilidade vertical em duas perspectivas, por software ou por hardware. A escalabilidade vertical por hardware consiste em adicionar ou remover mais poder de processamento e memória na máquina física rodando a aplicação. Da perspectiva de software, a escalabilidade vertical consiste em otimizar algoritmos e o código da aplicação.

Segundo (RAJAPAKSE, 2012) as principais desvantagens em escalar verticalmente um sistema via hardware é o custo dessa expansão e o inevitável tempo demandado para escalar. Este tempo para escalar ou até mesmo um mau funcionamento da máquina acaba afetando o funcionamento do sistema inteiro, pois, por estar em uma única máquina física, a disponibilidade de todo o sistema será comprometida.

Escalabilidade horizontal

Ao contrário da escalabilidade vertical que aumenta o poder de processamento de uma única máquina física, a escalabilidade horizontal consiste em adicionar novas máquinas. De acordo com (RAJAPAKSE, 2012) as principais vantagens da escalabilidade horizontal é a redução de custo, uma vez que mais máquinas com poder processamento inferior tem um custo menor do que adicionar mais processamento a uma única máquina. Além disso, tem-se a garantia de disponibilidade da aplicação, pois se uma máquina parar de funcionar, existem outras mantendo a aplicação disponível.

Ainda assim, este tipo de escalabilidade enfrenta algumas desvantagens caso o sistema não esteja preparado para funcionar sob um conjunto de máquinas. Uma destas é a necessidade de remodelar o sistema para permitir o paralelismo e a distribuição do trabalho em diversas máquinas, também chamado de balanceamento de carga.

2.1.3 Balanceamento de carga

Conforme um sistema é escalado horizontalmente, de modo que seus serviços fiquem distribuídos entre diversas máquinas, surge a necessidade de um mecanismo que distribuí as cargas de trabalho e delegue-as entre as máquinas do cluster.

Para melhor entender o funcionamento deste mecanismo, pode-se pensar no exemplo de um servidor web. O balanceamento de carga neste contexto, consiste em receber as conexões feitas por processos clientes e decidir com base em algum determinado critério, para qual servidor esta conexão deve ser encaminhada. Segundo (RAJAPAKSE, 2012), estes critérios também são chamados de ***estratégia de balanceamento de carga*** e os

mais comuns são:

- **Round Robin:** esta é a estratégia mais simples e tem o funcionamento da estrutura de dado do tipo fila (FIFO - First In First Out), onde os servidores revezam as conexões recebidas;
- **Número mínimo de conexões:** esta estratégia consiste em encaminhar as conexões para o servidor com menos carga de trabalho, de modo a evitar que os servidores fiquem sobrecarregados;
- **Tempo de resposta mais rápido:** esta estratégia parte do pressuposto de que o objetivo é minimizar o tempo de resposta. Sendo assim, a conexão é encaminhada para o servidor que responder mais rápido;
- **Ponderada:** esta estratégia leva em conta o cenário em que as máquinas não possuem a mesma capacidade de processamento ou memória, sendo assim, o servidor com maior capacidade receberá as conexões. A vantagem desta estratégia é que a ponderação é customizável.

2.1.4 Comunicação

Estabelecer um protocolo de comunicação é uma outra necessidade que surge com a escalabilidade horizontal, onde é definido que a aplicação será distribuída em diversas máquinas. Afinal, as máquinas presentes no cluster trabalharão em conjunto para manter a aplicação.

De acordo com (WETHERALL; TANEMBAUM, 2011), comunicação interprocessual é o coração de sistemas distribuídos e entre os modelos mais utilizados de comunicação de sistemas distribuídos estão ***Remote Procedure Call (RPC)*** e ***Message-Oriented Middleware (MOM)***. Cada um destes modelos possui regras de comunicação, também conhecidas como protocolos. Segundo (WETHERALL; TANEMBAUM, 2011), em muitos sistemas distribuídos, a comunicação não segue o estrito padrão de comunicação da interação cliente-servidor, sendo mais apropriado nestes casos, pensar em termos de mensagem como o sistema de email, por exemplo.

Remote Procedure Call (RPC)

Uma chamada procedimento é um sistema de comunicação comumente utilizado em sistemas distribuídos que permite a um processo chamar um procedimento em outro espaço de endereçamento de outro computador conectado pela rede. A diferença vital entre chamadas de procedimento local e remota é que na remota a chamada pode falhar

devido a possíveis problemas de rede. Habitualmente, este sistema é utilizado na implementação do modelo cliente-servidor em sistemas distribuídos. Segundo (WETHERALL; TANEMBAUM, 2011), a comunicação RPC consiste em uma máquina enviar uma chamada passando parâmetros ao servidor e aguardar uma resposta. O servidor sempre fica aguardando uma chamada e ao recebê-la, extrai os parâmetros para realizar algum tipo de processamento e retorna os resultados como resposta, e por fim volta a aguardar outras chamadas.

Por natureza, a comunicação RPC é ***síncrona***. Isto significa que o processo cliente aguarda a resposta de sua chamada antes de prosseguir com outras tarefas. No entanto, este tipo de comunicação pode não ser interessante em casos que a chamada não produza resultados ou quando o processo cliente não pode simplesmente esperar a chamada terminar. Tendo em vista estes problemas, foi criado a comunicação RPC ***assíncrona*** onde o processo faz a chamada e continua realizando outras tarefas enquanto a chamada é processada. A comunicação assíncrona possui alguns problemas como a não garantia de que a chamada foi enviada, sendo necessário o uso de outros mecanismos baseados em filas de mensagens como o Message-Oriented Middleware ou Middleware Orientado à Mensagens.

Message-Oriented Middleware (MOM)

De acordo com (WETHERALL; TANEMBAUM, 2011), a comunicação baseada em filas de mensagens consiste em aplicações comunicando entre si inserindo mensagens em filas específicas. Cada aplicação pode possuir filas de mensagens privadas ou dividir filas de mensagens com outras aplicações. A vantagem deste mecanismo é de que as mensagens enviadas irão para a fila de mensagens do recipiente. No entanto, não é garantido que a mensagem será processada ou quando.

Segundo (COULOURIS et al., 2013), os pontos positivos sobre comunicação com fila de mensagens são:

- Não há conexões diretas entre os processos;
- A comunicação entre processos é assíncrona;
- As mensagens podem ser enviadas ou recebidas de acordo com uma prioridade pré-definida.

E os pontos negativos:

- Há limites no número de filas, na capacidade individual das filas e no tamanho da mensagem;

- As filas de mensagens não são baseadas em conexão e o kernel não guarda referência dos processos para filas, sendo assim, é difícil saber quando a fila deve ser removida.

2.1.5 Consistência e Replicação

Uma outra necessidade que surge com a escalabilidade horizontal é a consistência e replicação dos dados. Com diversas máquinas trabalhando em uma mesma tarefa, é importante que todas possuam acesso aos mesmos dados. Isso significa que, quando uma cópia é atualizada, todas as demais devem ser atualizadas também.

De acordo (TANENBAUM; STEEN, 2017), os dois principais motivos para replicar dados é pela confiabilidade e a performance do sistema.

Segundo (TANENBAUM; STEEN, 2017), da perspectiva da confiabilidade, replicar os dados para manter a confiabilidade do mesmo é uma proteção contra dados corrompidos. Por exemplo, em um cenário com três cópias de um arquivo e cada operação de leitura e escrita é executada em todas as cópias. Em casos de corrupção de dados em uma das cópias, é possível checar a integridade desta analisando as outras cópias que não sofreram falhas.

Da perspectiva da performance, a replicação se faz necessária quando um sistema escala em números. Por exemplo, quando um crescente número de processos necessitam acessar informações mantidas por um único servidor. Neste casso, pode-se aumentar a performance replicando o servidor e dividindo a carga de trabalho.

Embora a replicação dos dados ajuda a manter a confiabilidade dos dados e aumenta a performance, de acordo com (TANENBAUM; STEEN, 2017), manter múltiplas cópias pode causar problemas de consistência. Ao atualizar uma cópia, as modificações devem ser propagadas para as outras cópias. No entanto, propagar as atualizações não garante que todas as cópias serão atualizadas na mesma hora. Por exemplo, quando um cliente acessa uma página web e a mesma demora para carregar dependendo da quantidade de acessos ao servidor. Replicando este servidor, e cada um com uma cópia daquela página web, permitirá que o cliente tenha uma resposta mais rápida ao acessar a página. Mas o problema ao propagar uma atualização da página web para as demais réplicas é que alguns clientes podem acabar acessando uma versão desatualizada daquela página.

Tendo definido conceitos como ***Arquitetura***, ***Escalabilidade***, ***Balanceamento de carga***, ***Comunicação*** e ***Consistência e Replicação*** em Sistemas Distribuídos, é necessário compreender as necessidades de um jogo online massivo e a aplicação dos conceitos de sistemas distribuídos . Sendo assim, na próxima seção serão abordados conceitos que envolvem o desenvolvimento de um jogo online.

2.2 Arquitetura de jogos online

Neste capítulo serão abordados características e arquiteturas de jogos online, suas necessidades, desafios e técnicas de solução. Uma característica muito importante presente em jogos e que em jogos online não é muito diferente é o game loop.

2.2.1 Game Loop

Segundo (NYSTROM, 2014), quando falamos de game loop, nos referimos ao ciclo de vida do processo do jogo. Sendo assim, o processo está rodando enquanto a condição imposta nesse loop não for quebrada. Geralmente, game loops ficam o tempo todo recebendo e tratando as entradas do usuário. Por exemplo, sempre que o usuário apertar uma tecla para seu personagem pular, um código dentro do game loop é responsável por renderizar na tela do jogador o pulo do personagem. Em jogos online não é muito diferente, a todo frame o cliente recebe informações do servidor e essas informações são renderizadas na tela do jogador. Em jogos que não são online, o jogo existe enquanto o usuário não decidir fechá-lo. Já em jogos online, o jogo não está rodando na máquina do usuário e sim em um servidor. Sendo assim, a execução do jogo é dita persistente pois é independente dos usuários conectados a ele.

2.2.2 Mundo Persistente

De acordo com (BARRON, 2001), quando um jogo online continua existindo independente de ter ou não jogadores online, ele é dito persistente. Há grandes diferenças entre jogos persistentes e jogos não persistentes. Em um jogo não persistente, a história e consequentemente o jogo, tem um começo, um meio e um fim. Já em jogos persistentes, o jogo não tem um fim. Isso é possível devido à natureza autoritativa de servidores de jogos online.

2.2.3 Servidor Autoritativo

Normalmente, os servidores de jogos online são arquitetados de modo que a simulação do jogo rode apenas no servidor. Assim, os clientes não atualizam nenhuma informação a respeito do estado do jogo. Os clientes apenas informam suas ações e todo o cálculo e lógica fica por conta do servidor, que replica as informações do estado do jogo para todos os clientes. E o cliente fica responsável apenas por renderedizar as informações recebidas. Um servidor organizado deste modo é chamado de servidor autoritativo.

Segundo (BARRON, 2001), o principal motivo dos servidores de jogos online serem autoritativos é devido ao fato de que os clientes não são confiáveis. Quando se trata de jogos que não são online, geralmente os desenvolvedores não procuram implementar técnicas que evitem as trapaças, isso porque independente das trapaças, outros jogadores

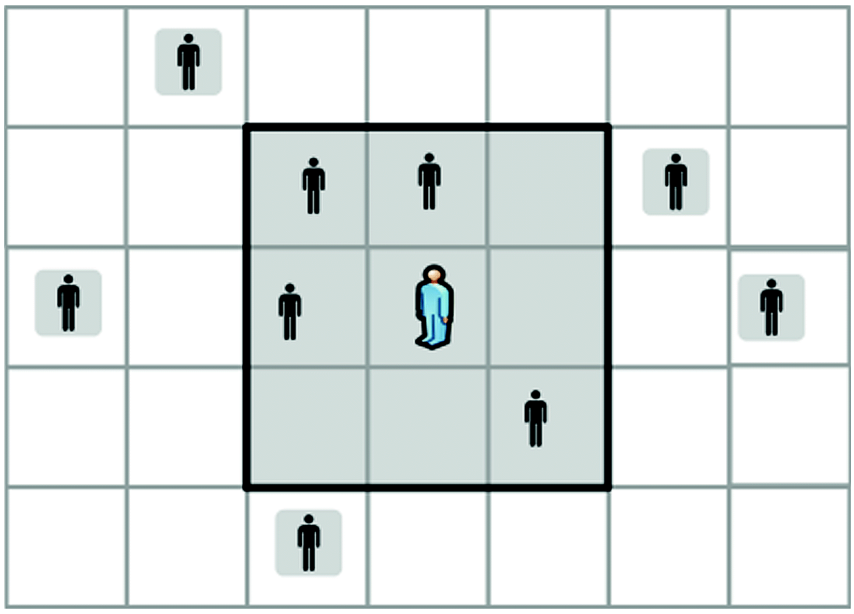
não são afetados. Já em jogos online, isso é uma grande preocupação, uma vez que a jogabilidade deve ser justa para todos os jogadores envolvidos. De acordo com (WENDEL, 2012), trapaça em jogos online pode ser definida como "uma atividade que modifica a experiência de jogo para dar vantagem a um jogador em cima de outros". Podemos pensar na movimentação do personagem como um bom exemplo. No modelo em que o servidor é autoritativo, todo o cálculo de coordenadas do personagem no mundo virtual é feito pelo servidor, cabendo ao cliente apenas indicar a ação feita pelo usuário. Já no modelo não-autoritativo, todo este cálculo que é feito no servidor, passa a ser feito no cliente. Com os cálculos sendo feitos nos clientes, ao serem enviados pelo servidor, é possível que usuários mal intencionados interceptem a informação e modifiquem de modo a beneficiarem a si próprios.

2.2.4 Gerenciamento de área de interesse

De acordo com (MORGAN, 2005), o gerenciamento de área de interesse tem o propósito de diminuir o tráfego de informações enviadas pelo servidor. Em um servidor que não possui este tipo de gerenciamento, todas as informações a respeito do mundo virtual são enviadas para todos os clientes, mesmo que estas informações não façam sentido para alguns. A quantidade de informações recebidas e enviadas pelo servidor passa a aumentar na medida em que o número de usuários aumenta. Levando em consideração que estas informações a respeito do mundo é requisitada a todo momento pelo cliente, podemos ver que há uma grande quantidade de informações trafegando. Há diversas técnicas neste tipo de gerenciamento, que é a subdivisão de servidores. O objetivo que estas técnicas tem em comum é de particionar o mundo virtual em diversas áreas, onde cada área tem um servidor responsável para gerenciá-la. Assim, o jogador recebe apenas informações da área em que está presente ou de áreas vizinhas. Esta técnica parte do pressuposto de que o jogador só está interessado nas informações que ele está vendo.

Atualmente, há dois tipos de subdivisão de servidores, a ***subdivisão estática*** e a ***subdivisão dinâmica***.

Figura 2 – Área de interesse



Fonte: Handbook of Multimedia for Digital Entertainment, 2009, Arts by Dewan Tanvir AhmedShervin Shirmohammadi

2.2.5 Subdivisão de servidores

Subdivisão estática

Segundo (SUMILA, 2011) uma funcionalidade comum presente em servidores de grande escala é a subdivisão do mundo do jogo em regiões geográficas, onde cada região e os jogadores presentes nesta região são gerenciados por um servidor.

Basicamente, há dois principais modos de se fazer a conexão de regiões pertencentes a servidores diferentes. O primeiro modo é baseado em portais. Isto significa que o jogador tem que passar por um portal para chegar a uma outra região e receber informações de eventos presentes naquela região. Geralmente a troca de servidores é mascarada com uma tela de carregamento. No segundo e último modo, o jogador não percebe a troca de servidores. O jogador simplesmente enxerga o mundo como uma região só.

Subdivisão dinâmica

De acordo (SUMILA, 2011), ambos os métodos de divisão estática apresentados anteriormente são suscetíveis às mudanças de concentração de jogadores. Ao longo do ciclo de vida de um MMO, é esperado que a concentração aumente devido a maturidade do mesmo ou aos eventos presentes no mesmo. Para garantir o bom funcionamento do servidor, é possível dividir o mundo com regiões de tamanhos diferentes, de acordo

com a densidade da população. Mas, uma vez que a tarefa de prever a quantidade de jogadores presentes em cada região e quais regiões estarão mais lotadas em relação as demais é difícil, se faz necessário o uso de uma técnica dinâmica. Esta técnica dinâmica consiste em reorganizar as regiões e seus tamanhos de acordo com a densidade da população.

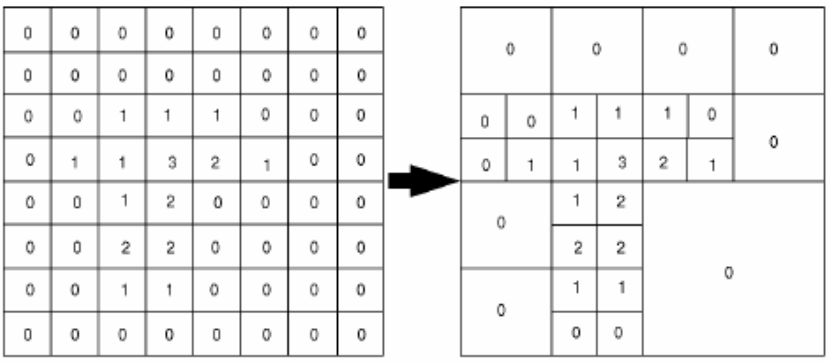
Ambos tipos de subdivisão de servidores utilizam algoritmos para particionar o mundo. Na próxima seção serão abordados os dois principais algoritmos mais comuns, o ***Quadtree*** e o ***kD-Tree***.

2.2.6 Algoritmos de particionamento

Quadtree

Segundo (LI; MUKUNDAN, 2013), Quadtree é um método de partição baseado em árvore que no ínicio de sua criação foi muito utilizado no processamento de imagem. Desde então, tem-se encontrado outras aplicações deste método em diversas outras áreas como ray tracing[0] e detecção de colisão. Basicamente, este método consiste em particionar hierarquicamente um espaço bidimensional, onde cada parte é chamada de quadrante ou nó. Cada nó dentro do quadtree tem exatamente quatro nós filhos e cada nó tem um volume máximo associado a ele.
O método quadtree utiliza menos memória em casos em que os jogadores não estão agrupados em uma única região e sim distribuídos pelo mundo. De acordo com (LI; MUKUNDAN, 2013) uma desvantagem deste método é a necessidade de reconstruir a árvore do quadtree para garantir o dinamismo do mesmo.

Figura 3 – Geração do quadtree a partir de uma grade 2D



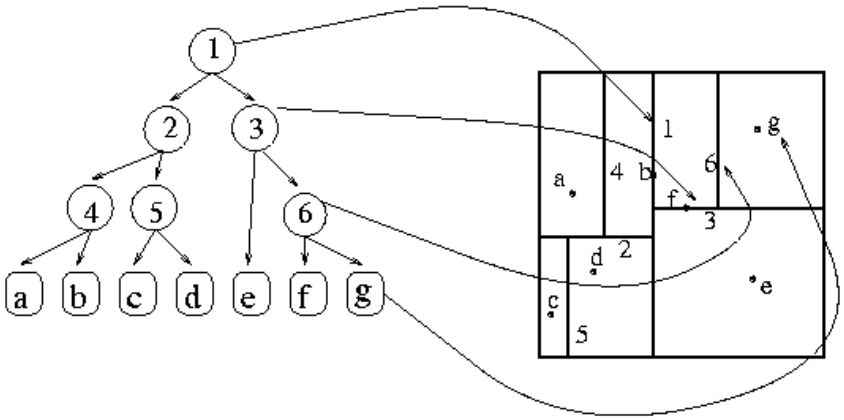
Fonte: Interactive Isosurface Ray Tracing of Large Octree Volumes, ResearchGate, 2006, página 2

[0] Ray Tracing: algoritmo de computação gráfica usado para renderização de imagens tridimensionais

kD-Tree

De acordo com (LI; MUKUNDAN, 2013), kD-Tree é um caso especial de árvore binária que divide um espaço k-dimensional hierarquicamente. A construção da árvore é feita dividindo um espaço não-vazio e seus subespaços utilizando o algoritmo de corte mediano recursivo. Segundo (BLOOMBERG, 2018), o algoritmo do corte mediano é um algoritmo utilizado para classificar dados de um número arbitrário de dimensões em séries de conjuntos, cortando recursivamente cada conjunto de dados no ponto mediano ao longo da dimensão mais longa. Utilizando este algoritmo, o kD-Tree primeiro utiliza a coordenada no eixo x, depois no eixo y, e novamente no eixo x, e assim sucessivamente. A profundidade da árvore é determinada quando as folhas da árvore possuem apenas um número máximo de objetos especificados ou quando a profundidade atingiu o limite máximo.

Figura 4 – Geração do kD-Tree



Fonte: kD-Tree, MIT Computer Science Artificial Intelligence Lab, http://groups.csail.mit.edu/graphics/classes/6.838/S98/meetings/m13/kd.html

2.3 Medição de performance de servidores

Quando o desempenho de uma rede cai, se faz necessário o uso de técnicas de medição de performance para determinar qual o problema da rede e o que está causando o mau desempenho.

Segundo (WETHERALL; TANEMBAUM, 2011), as medições podem ser feitas de diferentes maneiras. A medição mais básica é a medição da **latência**, que consiste em cronometar o quanto uma atividade leva para ser concluída. A **latência** mede o tempo de processamento de ponta a ponta. Em um cenário de envio e recebimento de menagens, a latência pode ser determinada medindo o tempo entre o envio e o recebimento de uma mensagem, técnica também conhecida como roundtrip. A latência é medida na máquina

do cliente, onde é levado em consideração a sobrecarga da rede. Outros modos consistem em contadores para registrar quantas vezes um evento ocorreu. No entanto, geralmente se está interessado na quantidade de bytes processados em um determinado intervalo de tempo, também conhecida como **taxa de transferência**.

A **taxa de transferência** ou **throughput** mede a quantidade de mensagens processadas pelo servidor em um intervalo de tempo específico (por exemplo, por segundo). Segundo, (IWRINGER, 2012), a taxa de transferência pode ser determinada medindo o tempo gasto para processar um conjunto de mensagens e em seguida utilizando a seguinte equação:

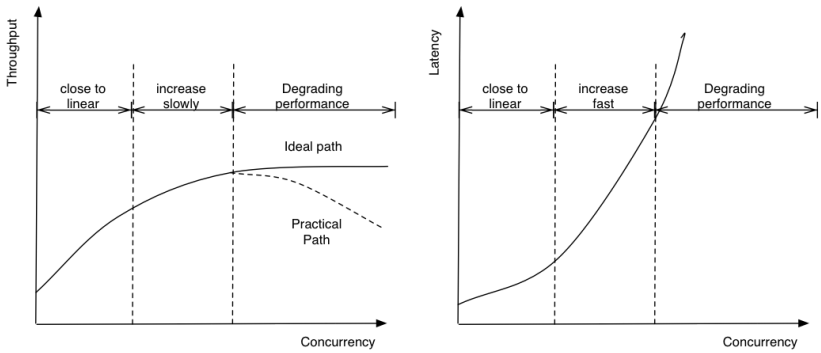
$$T = \frac{N}{t} \tag{2.1}$$

Onde **T** é a taxa de transferência, **N** é o número de solicitações concluídas e **t** é o tempo total gasto para concluir as solicitações.

De acordo com (IWRINGER, 2012), diferentemente da medição de capacidade (por exemplo, velocidade de processamento da CPU), que é uma medição estática, a medição de performance é dinâmica. A medição de performance é influenciada pela concorrência e pelas unidades de trabalho. Um tamanho maior de unidades de trabalho tem uma influência negativa na latência e na taxa de transferência. A concorrência é o número de unidades de trabalho agregadas (por exemplo, mensagens) processadas em paralelo (por exemplo, por segundo). Uma concorrência alta tende a aumentar a latência (tempo de espera) e diminuir a taxa de tranferência (unidades processadas).

Para visualizar o desempenho do servidor, geralmente é desenhado um gráfico de latência ou taxa de transferência em relação ao tamanho da unidade de trabalho, conforme pode ser visualizado na figura 5 abaixo:

Figura 5 – Characteristic Performance Graph's of a Server



Fonte: https://www.slideshare.net/hemapani/characteristic-performance-graph-of-a-server

2.4 Trabalhos relacionados

Nessa seção serão apresentados os trabalhos relacionados relevantes ao presente trabalho.

2.4.1 A Distributed Architecture for Massive Multiplayer Online Role-Playing Games

(ASSIOTIS; TZANOV, 2006) apresentam uma nova abordagem utilizando a arquitetura cliente/servidor com a abordagem distribuída para MMORPG's, que consiste em dividir o mundo virtual em pequenas áreas tirando vantagem de área de interesse para reduzir a largura de banda tanto do servidor quanto dos clientes.

O trabalho também propõe uma solução para os problemas de interação entre jogadores residindo em áreas diferentes, gerenciadas por servidores diferentes. Para validarem suas soluções e mostrar os benefícios da abordagem distribuída em relação a não-distribuída foi criado um jogo 3D.

Tanto o servidor quanto o jogo de exemplo foi desenvolvido na linguagem de programação Java.

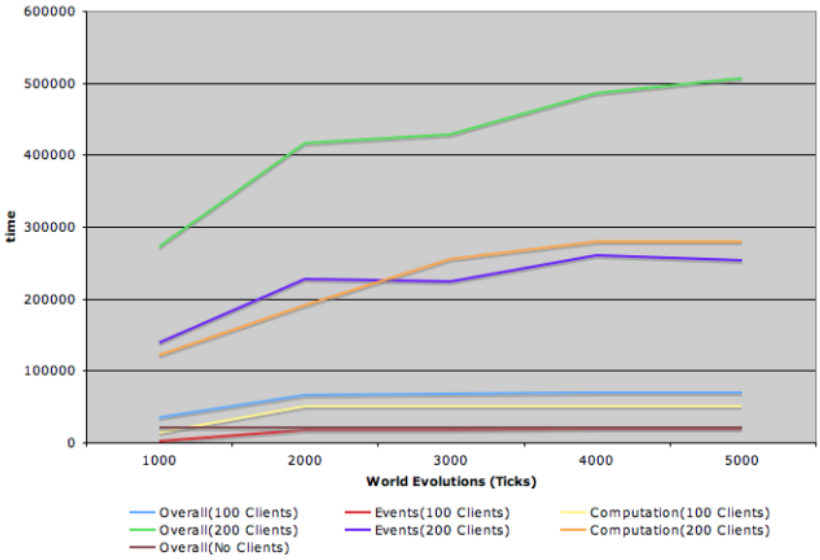
A validação do trabalhou consistiu no uso da plataforma Emulab para a simulação de uma topologia de redes de múltiplos servidores e clientes. Os clients foram configurados com latências randômicas de 10ms a 200ms e a taxa de perde de pacotes entre 0 e 0,3%. A configuração do hardware foi de 3GHz Pentium IV com 1GB de RAM e o sistema operacional Red Hat Linux 9.0. Em seus resultados, apresentam o sucesso das soluções apresentadas no trabalho, como também o ganho de performance ao utilizar a abordagem distribuída. É possível ver a comparação nos gráficos abaixo:

A figura 6 mostra os resultados do trabalho utilizando apenas um servidor, no qual é possível ver que esta abordagem não escala muito bem. Já na figura 7 é possível ver os resultados com a abordagem distribuída que foi capaz de aguentar o dobro de jogadores, utilizando apenas 3 servidores.

2.4.2 Distributed Architectures for Massively-Multiplayer Online Games

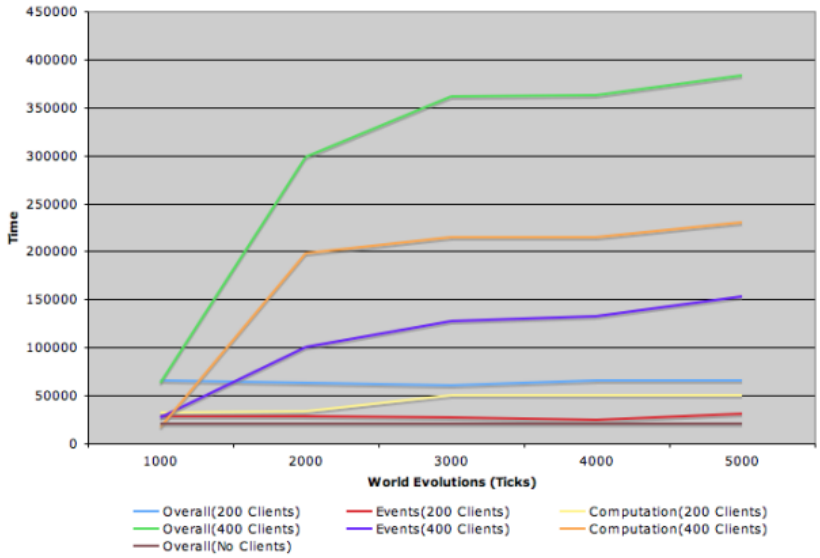
(GAUTHIERDICKEY, 2014) apresenta e compara arquiteturas utilizadas no desenvolvimento de servidores de MMO e a base teórica necessária sobre sistemas distribuídos, para que os leitores consigam compreender a abordagem distribuída em servidores de MMORPG. O autor também apresenta dados de pesquisa como latências toleradas por jogadores em diferentes gêneros de jogos.

Figura 6 – Solução não-distribuída



Fonte: A Distributed Architecture for Massive Multiplayer Online Role-Playing Games - (ASSIOTIS; TZANOV, 2006) Figure 4

Figura 7 – Solução distribuída



Fonte: A Distributed Architecture for Massive Multiplayer Online Role-Playing Games - (ASSIOTIS; TZANOV, 2006) Figure 5

3 Desenvolvimento

O objetivo deste trabalho é a aplicação da técnica de particionamento do mundo baseada na área de interesse, a fim de obter um servidor escalável e com um melhor gerenciamento das informações trafegadas entre servidores e entre servidor e clientes. Para isto foram desenvolvidos servidores seguindo a abordagem distribuída e não distribuída e o cliente a ser utilizado como exemplo.

Tanto o servidor quando o cliente foram desenvolvido utilizando a linguagem de programação Python 3.6. E para o desenvolvimento do cliente, foi utilizada a biblioteca gráfica PyGame na versão 1.9.6. PyGame é uma biblioteca gráfica de código aberto para jogos multiplataforma, disponível para Windows, Linux e macOS.

Para o compartilhamento e replicação dos dados entre os servidores, foi utilizado o MongoDB na versão 4.0. É um software de banco de dados NoSQL, de código aberto e orientado a documentos. O MongoDB utiliza documentos semelhantes a JSON com esquemas.

Os servidores foram hospedados na Amazon AWS, em máquinas com as seguintes configurações:

- **Nome:** Amazon Linux AMI 2018.03.0 (HVM), SSD Volume Type
- **Tipo:** t2.micro
- **vCPUs:** 1
- **Memória (GiB):** 1
- **Armazenamento (GiB):** 8
- **Tipo:** t2.micro
- **Localização:** Ohio - Estados Unidos

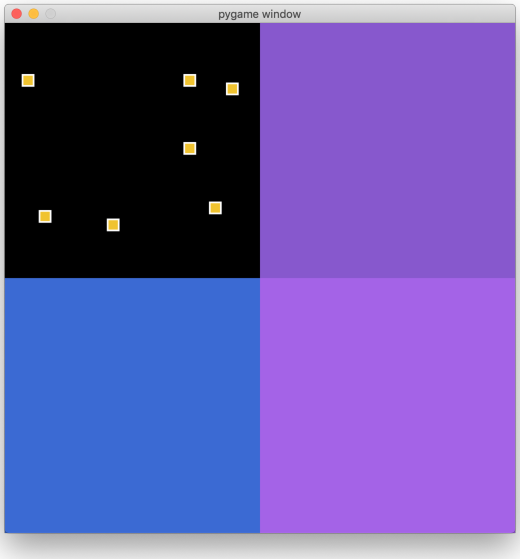
O desenvolvimento foi divido em 3 (três) fases, sendo elas:

3.1 Primeira fase - Cliente

Na primeira fase do desenvolvimento do presente trabalho, foi implementado um cliente para ser utilizado de exemplo, com uma mecânica básica e que engloba os aspectos principais de um jogo online massivo, de forma que seja possível validar a arquitetura a ser apresentada nos tópicos a seguir.

O cliente consiste em um jogo 2D Top-Down e sua mecânica básica envolve uma movimentação bidimensional e a possibilidade de atirar projéteis em ângulos de 0 a 360º. Os jogadores podem andar livremente pelo mundo virtual. Na abordagem distribuída, os jogadores podem mudar de área atravessando a "fronteira"entre áreas. Ao entrar em uma nova área, sua cor é modificada. O cliente de exemplo pode ser visualizado na figura 8.

Figura 8 – Cliente



Fonte: Autor

3.2 Segunda fase - Servidor não-distribuído

Na segunda fase deste projeto, foi desenvolvido o servidor do jogo seguindo a abordagem não distribuída e com o uso da arquitetura cliente/servidor. O uso da arquitetura cliente/servidor pode ser justificado de acordo com (ASSIOTIS; TZANOV, 2006) que mostra em seu trabalho que a arquitetura oferece maior segurança devido a centralização das informações do jogo no servidor. Ao final do desenvolvimento, foram coletados a média de latência e da taxa de transferência para a comparação com a abordagem distribuída.

3.3 Terceira fase - Servidor distribuído

Na terceira e última fase, foi desenvolvido o servidor proposto neste presente trabalho. Este servidor segue a abordagem distribuída e faz o uso da arquitetura cliente/servidor. O uso da arquitetura cliente/servidor se fez necessário, uma vez que o objetivo do trabalho é obter uma melhor performance sem sacrificar a segurança e a centralização das informações do jogo.

Além de segurança e eficiência é importante que o sistema seja arquitetado de modo que escale conforme o número de jogadores aumentar. Neste contexto, a escalabilidade que mais faz sentido é a escalabilidade horizontal, pois, além de menor custo, a disponibilidade do sistema é garantida pelo fato de mesmo que uma máquina pare de funcionar, outras estarão disponíveis. A escalabilidade vertical não é interessante pelo fato de que caso a máquina pare de funcionar, todo o sistema é comprometido. E isto, afeta um dos requisitos de um jogo online, que é estar disponível sempre.

Ao adotar a estratégia de escalar horizontalmente, é imprescindível o uso de uma estratégia de balanceamento de carga ou divisão de cargas de trabalho, pois o software deve estar preparado para funcionar em um conjunto de servidores. A estratégia de balanceamento de carga neste trabalho parte do pressuposto de que o jogador está interessado em enxergar apenas aquilo que está em seu raio de visão (AOI - Area of Interest ou Área de Interesse), ou seja, eventos acontecendo próximo a ele. Partindo deste princípio, a abordagem a ser seguida neste projeto é de particionar o mundo virtual em pequenas áreas, onde cada uma destas terá um servidor responsável para gerenciá-la.

Segundo (ASSIOTIS; TZANOV, 2006), seguindo esta abordagem de particionar o mundo virtual, pode-se encontrar alguns desafios como:

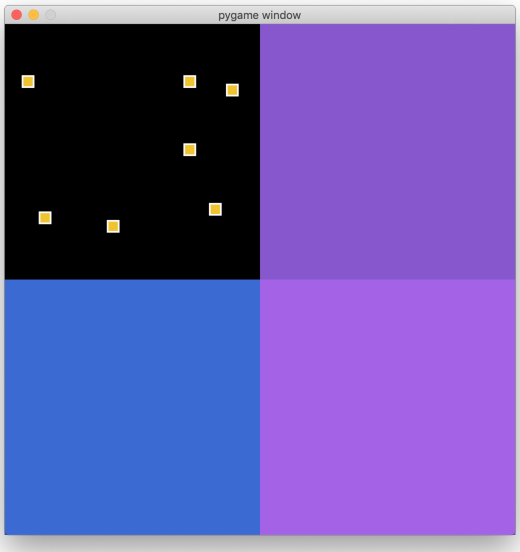
- **Área de interesse:** os jogadores devem receber apenas eventos ocorrendo próximos a eles.
- **Troca de servidores:** ao se deslocar de uma área para outra, o jogador deve ser conectado ao servidor correspondente à nova área, sem perceber que ocorreu a troca de servidores.
- **Bordas das áreas:** os eventos podem ocorrer na fronteira de duas áreas.

Tendo em vista esses desafios, se fez necessário definir como será a sincronia entre os servidores de zona. Os servidores se comunicarão através de mensagens e todos terão acesso ao servidor de banco de dados para fazer operações de leitura e escrita dos jogadores.

Por exemplo, quando um jogador cruzar a fronteira de outro servidor, o servidor responsável por este jogador deve notificar através de mensagens ao servidor responsável pela área que o jogador entrou. Após o aviso, o novo servidor passa a gerenciar este jogador.

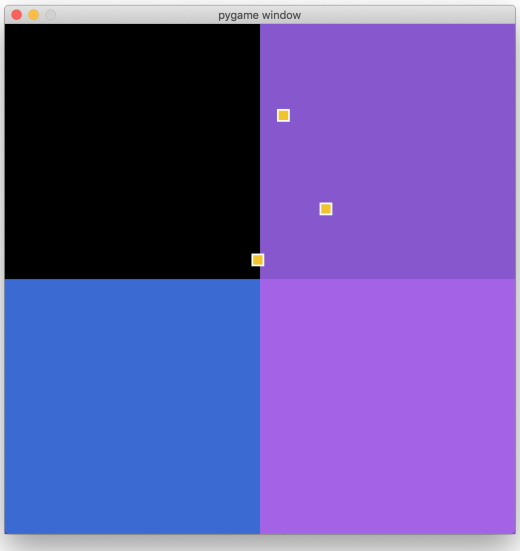
Após definido as tecnologias a serem utilizadas e os desafios a serem enfrentados, na próxima seção será apresentada a arquitetura proposta.

Figura 9 – Zona 1



Fonte: Autor

Figura 10 – Zona 2



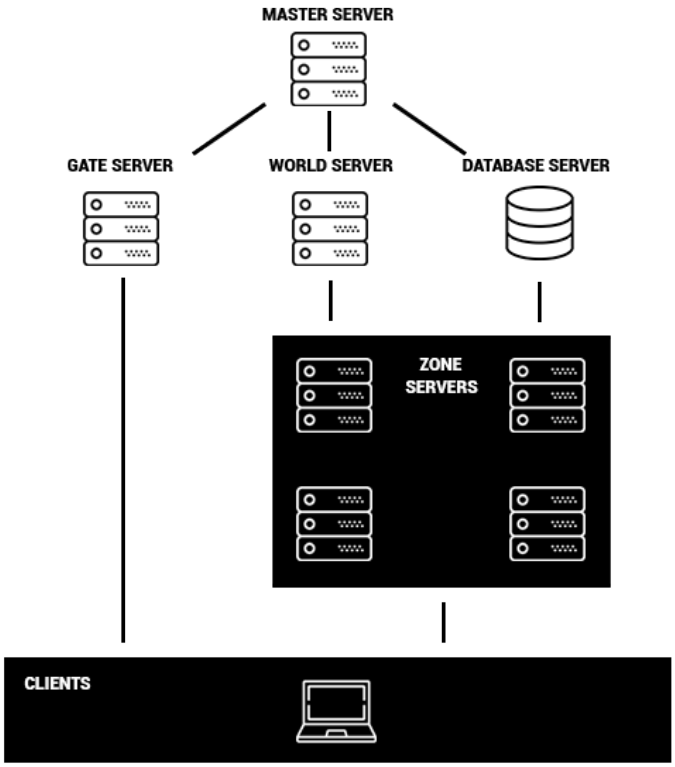
Fonte: Autor

3.4 Arquitetura proposta

De modo a obter um servidor escalável e com um melhor gerenciamento de informações trafegadas, é proposta uma arquitetura com 5 componentes, sendo eles: **Master-Server**, **Gate-Server**, **World-Server**, **Database-Server** e **Zone-Servers**. A organização da arquitetura pode ser visualizada na figura 11.

Toda comunicação entre os componentes presentes na arquitetura é feita através do envio de mensagens no formato JSON.

Figura 11 – Arquitetura



Fonte: Autor

Client: É instalado na máquina do jogador, responsável por enviar uma mensagem de conexão ao servidor **Gate**. Recebendo como resposta, em caso de sucesso, as informações referentes ao servidor de zona que deve se conectar. Também é responsável por enviar comandos de interação feito pelos usuários e renderizar informações recebidas do servidor de zona.

Gate-server: É o servidor responsável por receber as mensagens de conexões dos clientes, validar se todas informações necessárias foram enviadas pelos clientes e só então repassar os pedidos de conexão ao servidor mestre. Os pedidos de conexão são analisados com o uso do Design Pattern Publish-subscriber, onde possui 1 (um) publisher e (três) subscribers. O publisher é responsável por receber os pedidos de conexão e colocar em uma fila (Queue), enquanto os três workers revezam a análise dos pedidos de conexão presentes na fila.

Master-server: O servidor mestre possui informações sobre o mundo e suas zonas. É

Figura 12 – Filas (Queue) - Python

```python
from queue import Queue
from threading import Thread
import time

num_workers = 2
main_queue = Queue()

items = [0, 1, 2, 3, 4, 5]

def job(i, q):
    while True:
        item = q.get()
        print("Worker " + str(i) + " started working on " + str(item))
        time.sleep(5)
        q.task_done()

for i in range(num_workers):
    worker = Thread(target=job, args=(i, main_queue,))
    worker.setDaemon(True)
    worker.start()

for item in items:
    print("Queuing: " + str(item))
    main_queue.put(item)

main_queue.join()
```

Fonte: Autor

responsável por receber os pedidos de conexões encaminhados pelo gate-server, validar se o jogador existe no banco de dados, checar em qual servidor de zona o cliente deve se conectar e retornar uma resposta ao cliente atráves do gate-server.

Database-Server: É o servidor do banco de dados, onde as informações referentes aos jogadores e ao estado do jogo são salvas. O software de banco de dados utilizado foi o MongoDB, que utiliza um formato de documentos muito semelhante ao JSON.

Figura 13 – Estrutura do objeto Player - JSON

```json
{
    "login":"ricardo",
    "position":{
        "x":10,
        "y":10
    },
    "color":[
        255,
        255,
        255
    ]
}
```

Fonte: Autor

World-Server: O servidor do mundo é responsável por gerenciar os servidores de zona, delimitando e estabelecendo as áreas em que cada servidor de zona deve atuar. O world-server também recebe avisos dos servidores de zona de que o estado do jogo foi atualizado em alguma determinada área, cabendo a ele avisar aos outrros servidores que possivelmente foram afetados para que possam atualizar seus respectivos clientes.

Zone-Server: o servidor de zona é responsável por gerenciar os clientes presentes em sua área previamente delimitada pelo world-server, recebendo as ações do jogador, salvando no banco de dados e avisando ao servidor do mundo que fez mudanças. Ao mudar o estado do jogo, o servidor de zona também é responsável por manter seus clientes atualizados.

Entidades: as entidades presentes no mundo podem ser tanto os objetos jogadores, quanto os objetos não-jogadores (NPCs). A característica principal das entidades é a possibilidade de interagir com o mundo ou com outros jogadores e causar eventos no jogo.

Eventos: é definido como evento toda e qualquer ação ou acontecimento que ocorre no mundo virtual, que pode causar efeitos temporários ou permanentes no mundo. Dentre esses efeitos temporários, temos como exemplo: movimentação de entidades, ataques de entidades, entre outros. Já nos efeitos permanentes, temos como exemplo a destruição de alguma parte do mundo.

4 Resultados

Os testes foram dividos em 2 (duas) partes. A primeira consiste em coletar a média de latência e taxa de transferência no servidor com a abordagem não-distribuída. A segunda parte consiste em coletar a média de latência e taxa de transferência no servidor com a abordagem distribuída.

Ao final, é comparado os resultados de ambar abordagens e mostrado a relação da taxa de transferência com a latência.

A máquina utilizada para abrir diversos processos clientes possui a seguinte configuração:

Configurações:

- **Sistema:** macOS Mojave (10.14)
- **Processador:** 2,7 GHz Intel Core i5
- **Memória:** 8 GB 1867 MHz DDR3
- **Componente gráfico:** Intel Iris Graphics 6100 1536 MB

Internet

- **Download:** 60.39 Mbps
- **Upload:** 5.75 Mbps

Os testes foram realizados com uma quantidades de 3 à 50 jogadores concorrentes distribuídos pelo mundo virtual, que simulavam eventos de movimentação e disparo de projéteis em outros jogadores próximos.

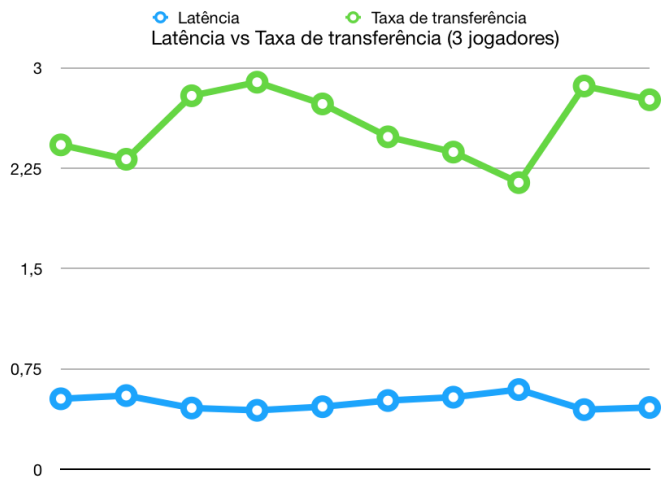
4.1 Primeira parte

Teste realizado no servidor com a abordagem não distribuída, na qual um único servidor é responsável por aceitar conexões de clientes, atualizar o estado do jogo e replicar para os clientes conectados.

Teste 1

- **Jogadores concorrentes:** 3

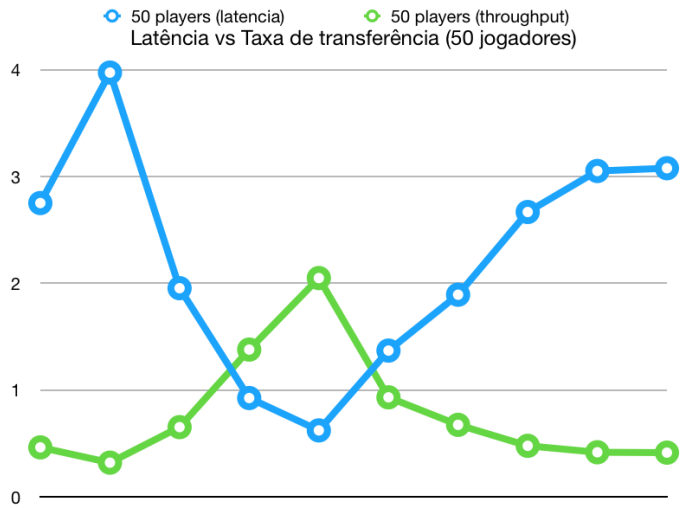
- **Latência média:** 0,501 segundos
- **Taxa de transferência média:** 2,578 kB/segundos.



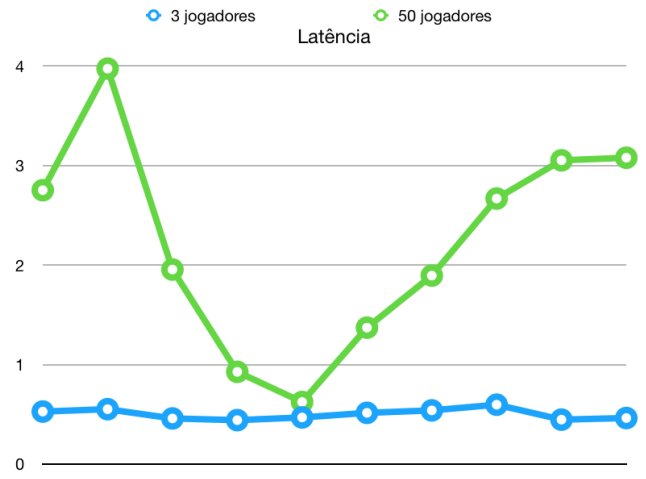
Fonte: Autor

Teste 2

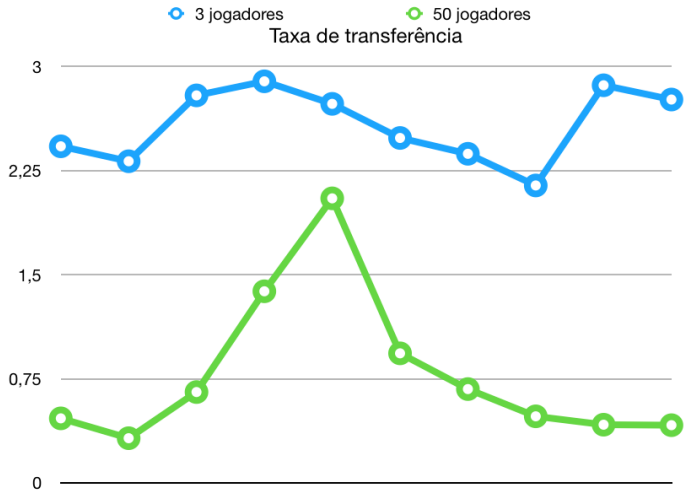
- **Jogadores concorrentes:** 50
- **Latência média:** 2,229 segundos
- **Taxa de transferência média:** 0,779 kB/segundos.



Fonte: Autor



Fonte: Autor



Fonte: Autor

4.2 Segunda parte

Teste realizado no servidor com a abordagem distribuída, na qual o mundo virtual foi dividido em 4 (quatro) servidores de zonas que são responsáveis por atualizar o estado do jogo e replicar para os clientes presentes em suas áreas de gerenciamento.

- **Servidores de zona:** 4
- **Jogadores concorrentes (total):** 50

Zona 1

- **Jogadores concorrentes:** 20

- **Latência média:** 0.874 segundos
- **Taxa de transferência média:** 1,972 kB/segundos.

Zona 2

- **Jogadores concorrentes:** 15
- **Latência média:** 0,602 segundos
- **Taxa de transferência média:** 2,100 kB/segundos.

Zona 3

- **Jogadores concorrentes:** 10
- **Latência média:** 0,451 segundos
- **Taxa de transferência média:** 2,327 kB/segundos.

Zona 4

- **Jogadores concorrentes:** 5
- **Latência média:** 0,304 segundos
- **Taxa de transferência média:** 2,475 kB/segundos.

4.3 Análise

Conforme visto na sessão 2.3 sobre medição de performance em servidores, a latência serve para medir o intervalo de tempo entre o envio de uma mensagem e o recebimento de uma resposta. Isso indica o tempo gasto para a mensagem chegar ao destino, ser processada e receber um resultado. Quanto mais latência, mais tempo demorará para uma atividade ser processada.

A latência também define a taxa de transferência máxima em uma comunicação entre servidores. Pensando em uma fila de mensagens que são enviadas de forma sequencial, quanto maior for a latência, maior será a demora para receber uma resposta e enviar a próxima mensagem. Sendo assim, a quantidade de mensagens enviadas (taxa de transferência) será menor durante essa comunicação.

Sendo assim, a latência tem uma relação com a taxa de transferência. Quanto maior a latência, menos dados serão enviados em um intervalo de tempo. E quanto menor a latência, mais dados são trafegados.

É possível observer essa relação nos resultados obtidos nos testes das duas abordagens. Na primeira abordagem, podem ser comparados os resultados obtidos com 3 jogadores e com 50 jogadores respectivamente. Com 3 jogadores concorrentes, a média de latência foi de 0.3 segundos e a taxa de transferência média de 0.400 kB/segundos. Aumentando o número de jogadores para 50, a média da latência subiu para 2.5 segundos e a taxa de transferência média caiu para 0.051 kB/segundos.

Nos testes da segunda abordagem, proposta pelo presente trabalho, é possível observar a mesma relação entre latência e taxa de transferência nos resultados. É possível notar que em zonas com quantidades maiores de jogadores concorrentes, a latência também é maior e a taxa de transferência também é menor. No entanto, diferente da primeira abordagem, nem todos jogadores sofrem com a alta latência e baixa taxa de transferência. Isso pode ser observado comparando os resultados na Zona 1 (com maior quantidade de jogadores) e na Zona 4 (com menor quantidade de jogadores).

Embora é possível notar uma melhora na latência e na taxa de transferência em zonas com pequenas quantidades de jogadores concorrentes, muitos jogadores aglomerados em uma única área, traz de volta o problema presente na primeira abordagem. Uma solução, conforme visto na sessão 2.2.5, é implementar uma subdivisão dinâminica no mundo virtual, onde através de um algoritmo de particionamento, é possível dividir as áreas dinâmicamente conforme a quantidade de jogadores aumentar naquela área, criando assim novos servidores gerenciando estas áreas.

5 Conclusão

Embora os resultados obtidos no presente trabalho apresentam uma melhora na latência e na taxa de transferência de dados ao particionar o mundo em zonas, nota-se que a melhora é obtida em casos em que os jogadores estão bem distribuídos pelo mapa. Em cenários onde um grande número de jogadores se aglomeram em uma única área, o problema apontado na abordagem não distribuída volta a aparecer. Uma possível solução é o uso de algoritmos de particionamento para realizar partições dinâmicas à medida em que os jogadores se conectam e se distribuem pelo mundo.

É possível notar também que a complexidade de desenvolvimento dos servidores é maior ao seguir a abordagem distribuída, uma vez que se faz necessário estabelecer protocolos de comunicação a serem utilizados pelos componentes da arquitetura, replicar os dados e o estado do jogo e tratar problemas referentes à troca de servidores e eventos entre servidores.

No entanto, os resultados obtidos neste trabalho ajudam a comparar as duas abordagens, conhecer suas vantagens, desvantagens e problemas de modo a obter um melhor entedimento de quando seguir cada abordagem.

Referências

ACHTERBOSCH, L.; PIERCE, R.; SIMMONS, G. Massively multiplayer online role-playing games: The past, present, and future. School of Information Technology and Mathematical Sciences, University of Ballarat, Australia, 2008. Citado na página 8.

ASSIOTIS, M.; TZANOV, V. A distributed architecture for massive multiplayer online role-playing games. NETGAMES, 2006. Citado 5 vezes nas páginas 9, 24, 25, 27 e 28.

BARRON, T. *Multiplayer Game Programming*. [S.l.]: Prima Publishing, 2001. v. 1. Citado na página 18.

BLOOMBERG, D. S. *Color quantization using modified median cut*. 2018. Disponível em: <http://www.leptonica.com/papers/mediancut.pdf>. Citado na página 22.

COMTEX. World of warcraft(r) subscriber base reaches 11.5 million worldwide. Blizzard, 2008. Citado na página 8.

COULOURIS, G. et al. *Sistemas Distribuídos - Conceitos e Projeto*. [S.l.]: Bookman Editora, 2013. v. 5. Citado 2 vezes nas páginas 12 e 16.

GAMESPOT. *Sword of the New World: Granado Espada QA - PC Previews at GameSpot*. 2007. Disponível em: <http://www.gamespot.com/pc/rpg/swordofthenewworld/news.html?sid=6170306&om_act=convert&om_clk=newsfeatures&tag=newsfeatures;title;3&mode=previews>. Citado na página 8.

GAUTHIERDICKEY, C. Distributed architectures for massively-multiplayer online games. Department of Computer Science - University of Oregon, 2014. Citado 3 vezes nas páginas 8, 9 e 24.

IWRINGER. *HOW TO MEASURE THE PERFORMANCE OF A SERVER*. 2012. Disponível em: <https://iwringer.wordpress.com/2012/05/03/how-to-a-measure-the-performance-of-a-server/>. Citado na página 23.

JOGOS, U. *Ragnarok Online (PC) - Análise*. 2011. Disponível em: <http://jogos.uol.com.br/analises/pc/ult398u191.jhtm>. Citado na página 8.

LI, B.; MUKUNDAN, R. A comparative analysis of spatial partitioning methods for large-scale, real-time crowd simulation. University of Canterbury, 2013. Citado 2 vezes nas páginas 21 e 22.

LIKARISH, P. et al. Demographic profiling from mmog gameplay. Dept of Computer Science - The University of Iowa and Palo Alto Research Corporation, 2011. Citado na página 8.

MORGAN, G. Scalable massively multiplayer online games. School of Computing Science, University of Newcastle upon Tyne, 2005. Citado na página 19.

NYSTROM, R. *Game Programming Patterns*. [S.l.]: Genever Benning, 2014. v. 1. Citado na página 18.

RAJAPAKSE, D. C. *A Fresh Graduate's Guide to Software Developtment Tools and Technologies*. [S.l.]: School of Computing - National University of Singapore, 2012. v. 2. Citado 3 vezes nas páginas 12, 13 e 14.

SUMILA, A. Visibility–based microcells for dynamic load balancing in mmo games. Queen's University, 2011. Citado na página 20.

TANENBAUM, A. S.; STEEN, M. V. *Sistemas Distribuídos: Princípios e Paradigmas*. [S.l.]: Pearson, 2017. v. 2. Citado 3 vezes nas páginas 11, 12 e 17.

VAULT, G. *Classic Game Postmortem: Meridian 59*. 2012. Disponível em: <https://www.gdcvault.com/play/1016676/Classic-Game-Postmortem-Meridian>. Citado na página 8.

WENDEL, E. Cheating in online games: A case study of bots and bot-detection in browser-based multiplayer games. Norwegian University of Science and Technology - Department of Telematics, 2012. Citado na página 19.

WETHERALL, D. J.; TANEMBAUM, A. S. *Redes de Computadores*. [S.l.]: Pearson, 2011. v. 5. Citado 4 vezes nas páginas 12, 15, 16 e 22.